ASD1-M1-10 · ALGORYTMY I STRUKTURY DANYCH I

Laboratorium haszowania

Haszowanie i rozwiązywanie kolizji

Autor: Daniel Zawisza

Numer indeksu: 93796

Aplikacja: Laboratorium haszowania

Semestr I · 2026/2027

Mini-projekt M1

Autorstwo i pomoc AI opisane na slajdzie 9.

02 · PROBLEM I CEL

Jedno hasło, różne kolizje

Funkcja $h(k)=k \bmod m$ kieruje klucz do komórki.

Różne klucze mogą trafić w to samo miejsce.

Cel laboratorium

Zrozumieć, jak łańcuchy i próbkowanie liniowe zachowują się przy wstawianiu, wyszukiwaniu, usuwaniu i przebudowie.

Odbiorca: student poznający tablice haszujące.

03 · ZAKRES

Jedna historia — dwie metody

- Implementacje łańcuchów i próbkowania liniowego
- Klucze 0–999, bez duplikatów, maks. 20 aktywnych
- Start m=7; próg α > 0,7; rozmiar 2m+1
- Interaktywny krok, Start/Pauza, tempo, reset, scenariusze
- Porównanie i 14 testów automatycznych

Sprawdzono również 5 przypadków działania interfejsu w lokalnym Chrome.

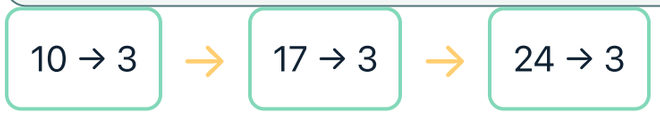
Rozwiązywanie kolizji

Łańcuchy

h(k) → koszyk → porównuj węzły → dopisz na końcu albo usuń właściwy węzeł.

Próbkowanie liniowe

h(k), h(k)+1, … (mod m). EMPTY kończy; DELETED zachowuje ścieżkę.



Przy przebudowie aktywne klucze wracają do nowej tablicy według kolejności dodania.

Stan, który można odczytać

Łańcuchy: tablica koszyków, każdy koszyk przechowuje listę.

Lista `order` zachowuje chronologię dodania dla rehashingu.

Próbkowanie: tablica rekordów `EMPTY`, `OCCUPIED`, `DELETED`.

Wstawianie pamięta `firstDeleted`, a sondowanie ma limit m kroków.

Wspólne zmienne: m, n, α=n/m, indeks bazowy, indeks sondowania i liczniki.

06 · TECHNOLOGIA I ARCHITEKTURA

Lokalnie i bez zależności online

HTML/CSS ↔ app.js: sterowanie i widok ↔ algorithms.js: generatory

JavaScript działa w przeglądarce; testy uruchamiane w Node.js. Nie używa zewnętrznych bibliotek ani API.

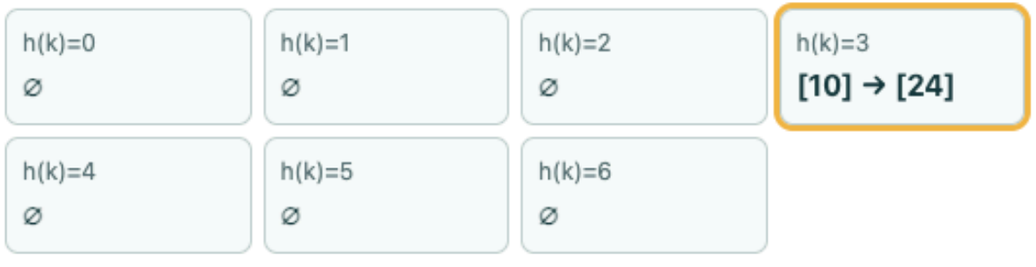
Wersja i liczba testów: patrz `results/tests.json` i `results/comparison.json`.

07 · INTERFEJS

Własny przebieg krokowy

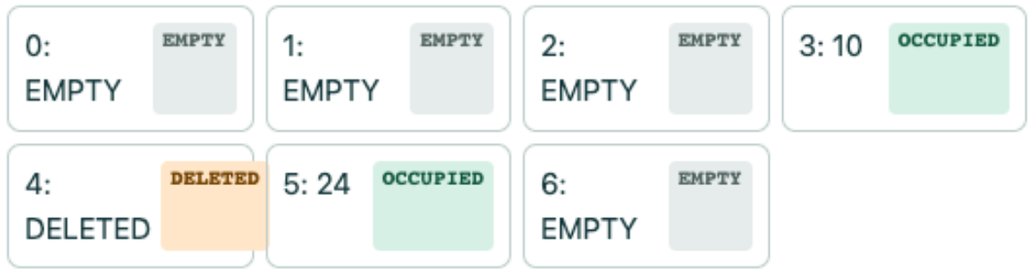
Łańcuchy

Kolizje trafiają do listy w koszyku. Wstawianie dopisuje na końcu. Przydatne w słownikach, gdy kolizje mogą tworzyć zmienne listy; ceną są dodatkowe węzły i możliwość długiego łańcucha.



Próbkowanie liniowe

EMPTY kończy wyszukiwanie; DELETED nie przerywa ścieżki. Tablica jest zwarta i ma dobrą lokalność pamięci, ale skupiska kluczy wydłużają sondowanie, a usuwanie wymaga tombstone.



Rys. 1. Wyszukiwanie 24 po usunięciu 17 — rzeczywisty zrzut, 26.10.2026.

08 · TESTY I PORÓWNANIE

Wyniki z lokalnego uruchomienia

14/14 testów PASS (Node v20.20.2, 2026-10-26).

Równomiernie: 8,16,24,32

Łańcuchy: 3 porównania / 3 odwiedziny

Próbkowanie: 6 / 11

Kolizje: 10,17,24,31

Łańcuchy: 11 porównań / 11 odwiedzin

Próbkowanie: 11 / 16

Każda historia obejmuje 2 wyszukiwania (trafione i nietrafione) oraz usunięcie. Rehash: osobno. Szczegóły w results/comparison.json.

09 · WNIOSKI I OGRANICZENIA

Co mówią liczniki?

- Rozkład kluczy wpływa na liczbę porównań i odwiedzin.
- Próbkowanie dobrze wykorzystuje lokalność, lecz skupiska wydłużają ścieżkę.
- Łańcuchy utrzymują kolizje w koszyku kosztem dodatkowych węzłów.
- To mała deterministyczna próbka, bez pomiaru czasu ściennego.

Autorstwo i AI: Autor projektu: Daniel Zawisza. Przy opracowaniu kodu, testów, wizualizacji i materiałów wykorzystano narzędzia AI: OpenClaw oraz Codex. Poprawność działania sprawdzono testami automatycznymi i kontrolą interfejsu; wyniki znajdują się w raportach dołączonych do projektu.

Źródło: Princeton Algorithms, Hash Tables; adres w dokumentacji.

10 · POKAZ PO SLAJDACH

Plan demonstracji produktu

1. Uruchom `index.html`, pokaż scenariusz kontrolny 10,17,24.
2. Wykonaj krok po kroku: usuń 17, znajdź 24 za DELETED.
3. Zmień własną historię i pokaż reset oraz tempo.
4. Porównaj dane równomierne i kolizyjne.
5. Pokaż przebudowę 7→15, raport testów i wyjaśnij pseudokod.

Po demonstracji — pytania i wyjaśnienie kodu.